ReFood Connect — Smart Food Waste Redistribution Platform

Comprehensive Project Report | Digital Assignment 3 (BCSE301L)

Academic Institution: Vellore Institute of Technology (VIT), Chennai Campus
Course Title & Code: Software Engineering (BCSE301L)
Student Team: Chirag Asija (24BDS1073) & Saanvi Gupta (24BDS1023)
Official GitHub Repository: https://github.com/ChiragAsija1323/ReFood-Connect
Evaluation Milestone: Review 3 (DA-3 Final Project Report & Appendices)

---

1. Problem Statement & Objectives

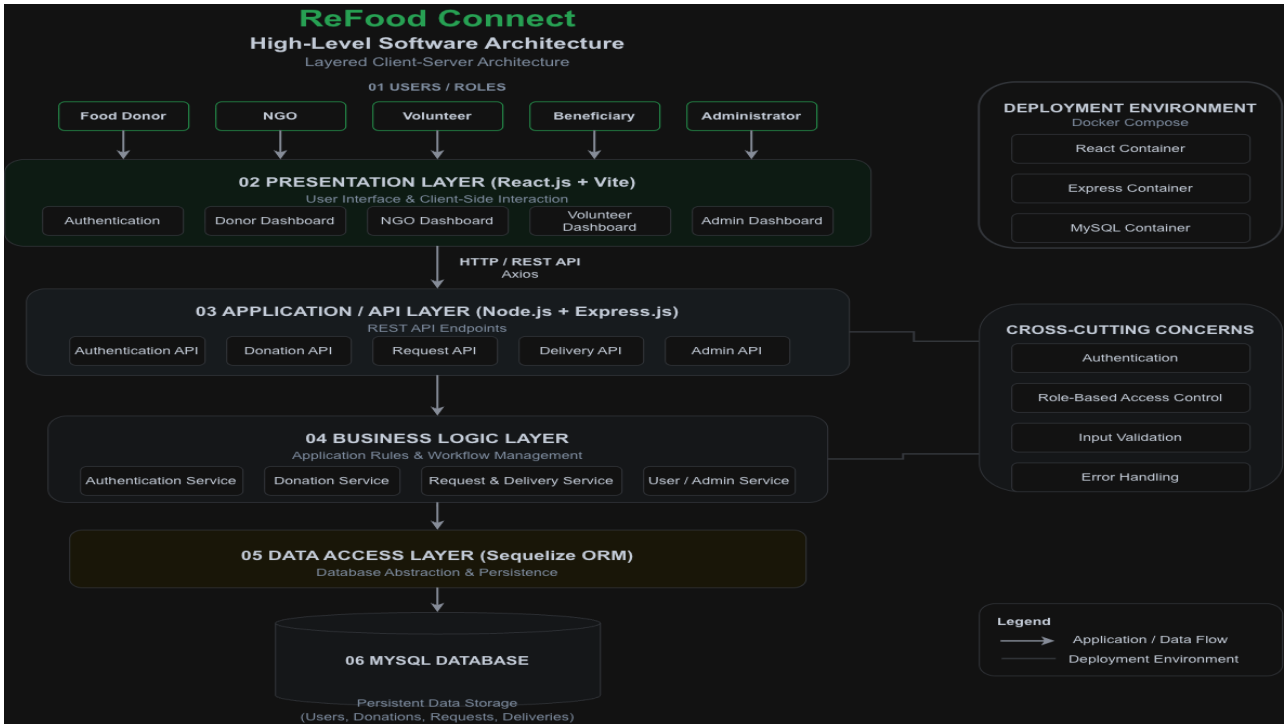
Every day, commercial food establishments discard edible surplus food due to event completion or overproduction, while shelters and underprivileged communities suffer from nutritional insecurity. Manual coordination via phone calls and messaging apps results in perishability failures and zero traceability. ReFood Connect automates the surplus redistribution lifecycle by unifying Food Donors, NGOs, Volunteers, and Administrators in an audited, role-governed platform.

2. User Stories Across Defined Personas

ID	Persona	User Story Summary	Priority
US-01	Donor	Register and log in securely with JWT issuance	Must Have
US-02	Donor	Post surplus batch with quantity and expiry timestamp	Must Have
US-03	NGO	Browse available listings and submit request claims	Must Have
US-04	Volunteer	Accept transport tasks and update transit to Delivered	Must Have
US-05	Admin	Access governance dashboard with real-time analytics	Must Have
US-06	All	Enforce salted password hashing and role access control	Must Have

3. System Architecture & Design Principles

ReFood Connect implements a Layered Client-Server Architecture utilizing the Service Layer Pattern. The React 19 frontend is decoupled from the Express 5 backend via a centralized Axios instance. Business logic is strictly contained within service classes (authService, donationService, adminService), while Sequelize ORM abstracts database persistence across MySQL (production) and SQLite (in-memory test isolation).



4. Design of Tests & Execution Results

The empirical test suite consists of 27 automated tests across 4 test suites:

```
• Integration Tests (IT-01 to IT-12): Registration, Login, Donation creation, NGO claiming, Volunteer transit, DB persistence.
• Regression Tests (RT-01 to RT-07): RBAC unauthorized rejection (403), Admin dashboard data integrity, invalid status transitions.
• Unit / Mutation Tests (UT-01 to UT-08): Direct service validation, empty string rejection, role-specific query filtering.
Result: 27/27 Tests Passed (100% Pass Rate) | Execution Time: 5.32 seconds
```

5. Appendices & Technical Design Artifacts

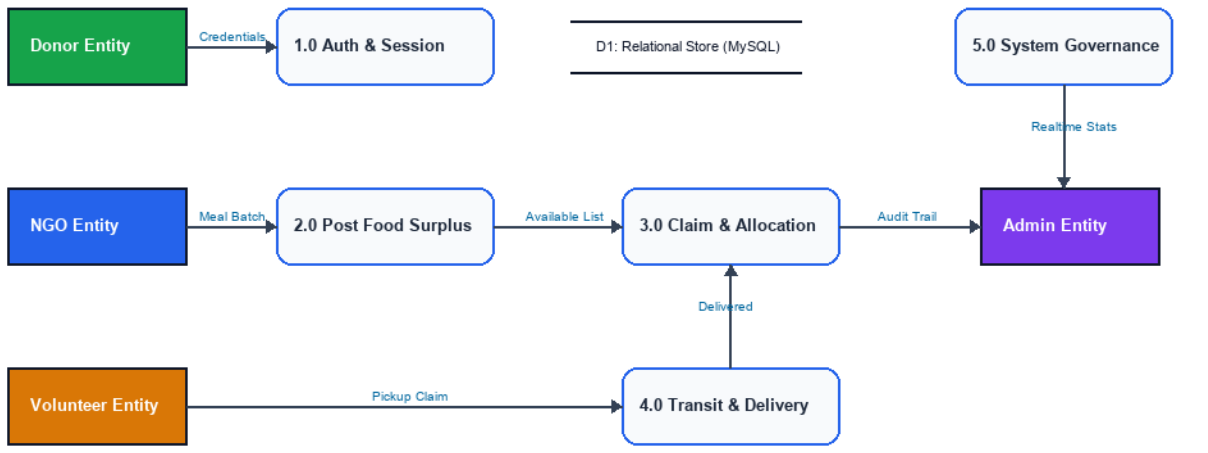
Appendix A: Software Requirements Specification (SRS)

Functional: Role-based JWT authentication; surplus batch creation with expiry validation; NGO request claiming; volunteer sequential transit tracking; admin analytics.
Non-Functional: Sub-second test execution via in-memory SQLite; bcrypt password hashing; SQL injection prevention via ORM; Docker containerization.

Appendix B: Data Flow Diagrams (DFD)

Level 0 & Level 1 Data Flow: Illustrates the decomposition of redistribution processes across entities, services, and the data store.

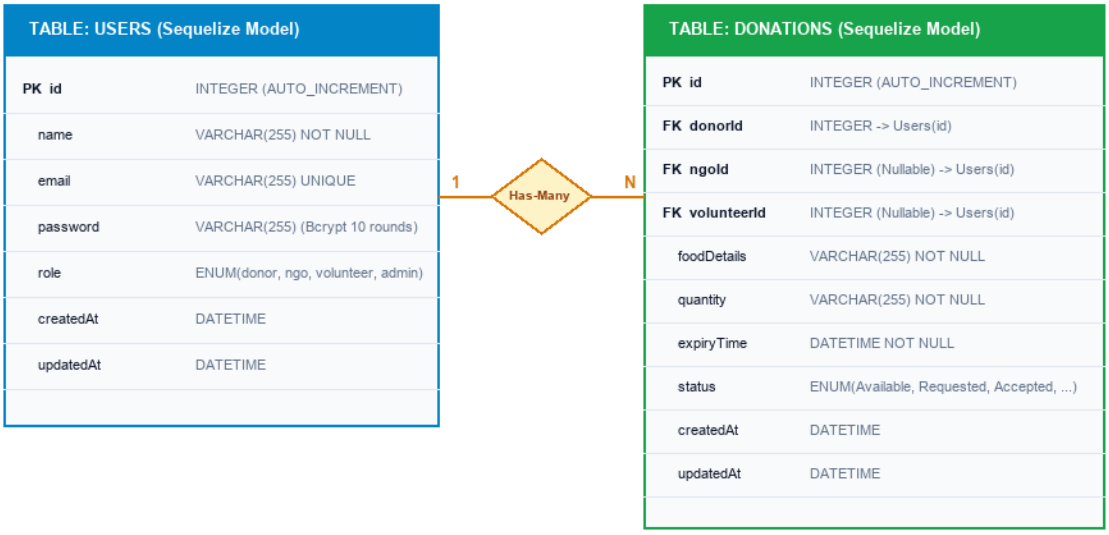
Figure B.1: Data Flow Diagram (DFD) - Level 1 Redistribution Lifecycle



Appendix C: Entity-Relationship Diagram (ERD)

Relational Schema: Users (PK id, role) associated via 1:N relations with Donations (FK donorId, ngoId, volunteerId).

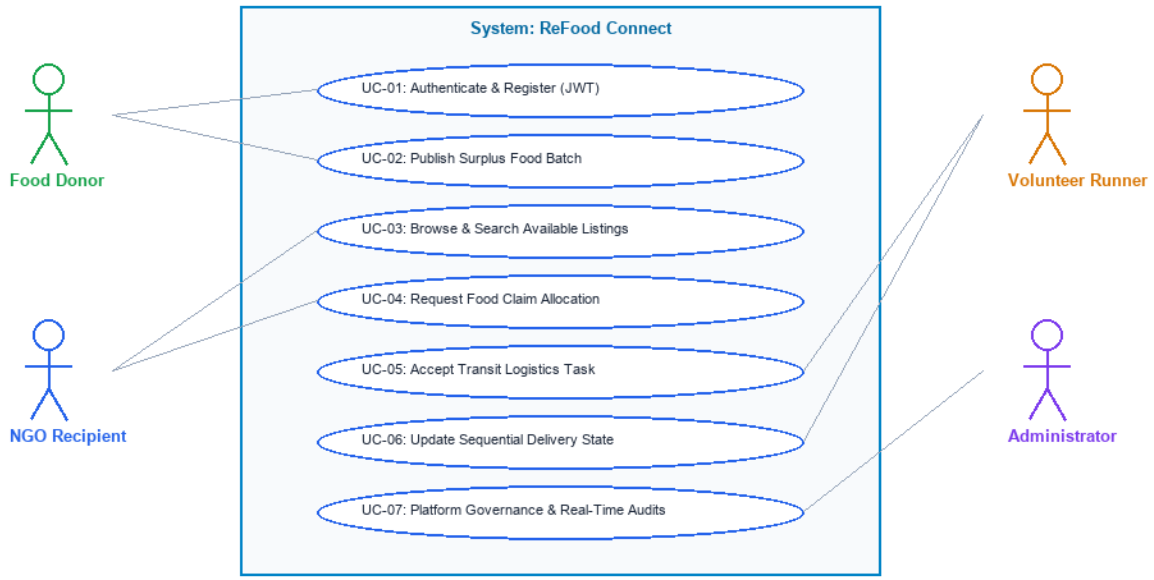
Figure C.1: Entity-Relationship Diagram (ERD) - Relational Model



Appendix D: Unified Modeling Language (UML) Diagrams

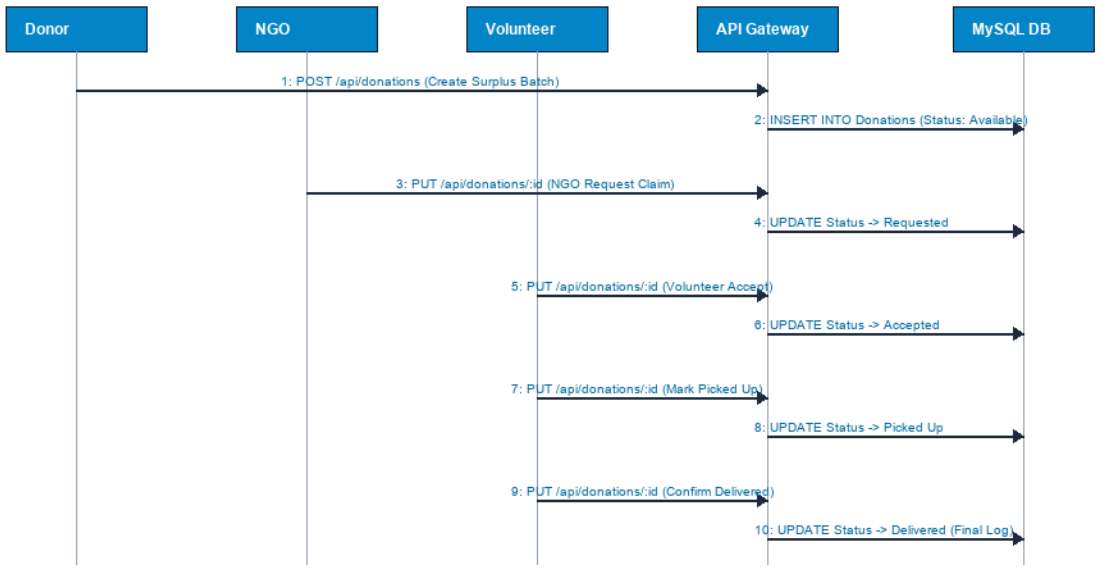
D.1 UML Use Case Diagram: System boundaries and actor interactions.

Figure D.1: UML Use Case Diagram - System Actors & Operational Goals



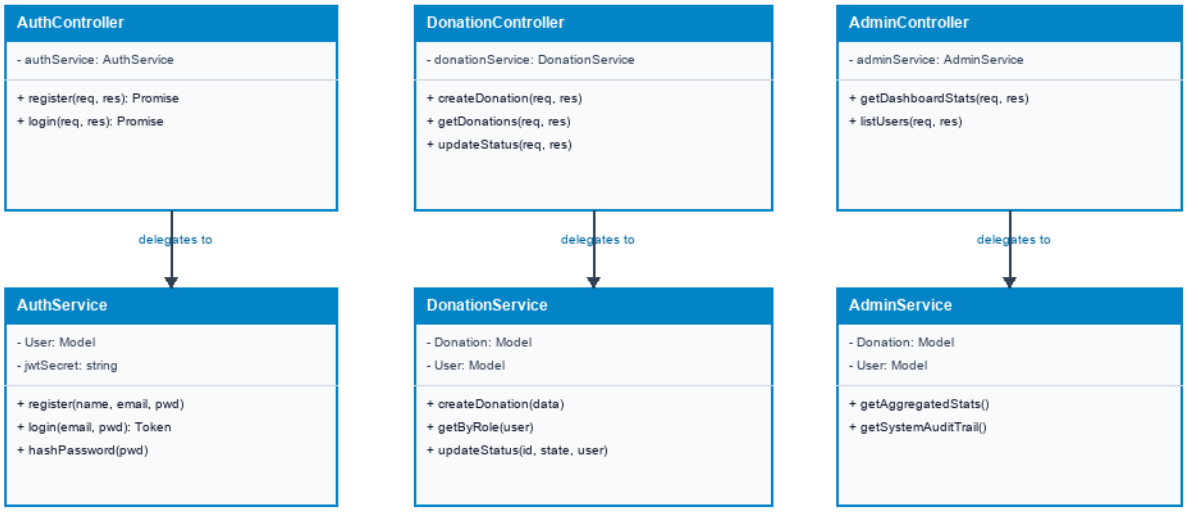
D.2 UML Sequence Diagram: End-to-end food donation lifecycle progression.

Figure D.2: UML Sequence Diagram - Sequential Redistribution Lifecycle



D.3 UML Class Diagram: Decoupled controllers, service layer, and models.

Figure D.3: UML Class Diagram - Layered Architectural Components



Appendix E: Code Listing & Repository Link

Official GitHub Repository: **https://github.com/ChiragAsija1323/ReFood-Connect**
All source code, Docker configs, test suites, and documentation are committed on branch main.